

4 属性识别

③表情检测

人脸表情识别 (Face expression recognition 简称FER)

普遍认为人类主要有六种基本情感：愤怒 (anger)、高兴 (happiness)、悲伤 (sadness)、惊讶 (surprise)、厌恶 (disgust)、恐惧 (fear)。而大多数表情识别是基于这六种情感及其拓展情绪实现的

主要困难点是：

- 1, 表情的精细化程度划分：每种情绪最微弱的表现是否需要被分类，分类的界限需要产品给出评估规则。
- 2, 表情类别的多样化：是否还需要补充其他类别的情绪，六种情绪在一些场景下远不能变现人类的真实情绪。因此除了基本表情识别外，还有精细表情识别、混合表情识别、非基本表情识别等细致领域的研究。
- 3, 缺少鲁棒性

4 属性识别

③数据集介绍

FER13 Dataset



Fer2013人脸表情数据集由35886张人脸表情图片组成，其中，测试图（Training）28708张，公共验证图（PublicTest）和私有验证图（PrivateTest）各3589张，每张图片是由大小固定为48×48的灰度图像组成，共有7种表情，分别对应于数字标签0-6，具体表情对应的标签和中英文如下：

0 anger 生气； 1 disgust 厌恶； 2 fear 恐惧； 3 happy 开心； 4 sad 伤心； 5 surprised 惊讶； 6 normal 中性。

<https://www.kaggle.com/deadskull7/fer2013>

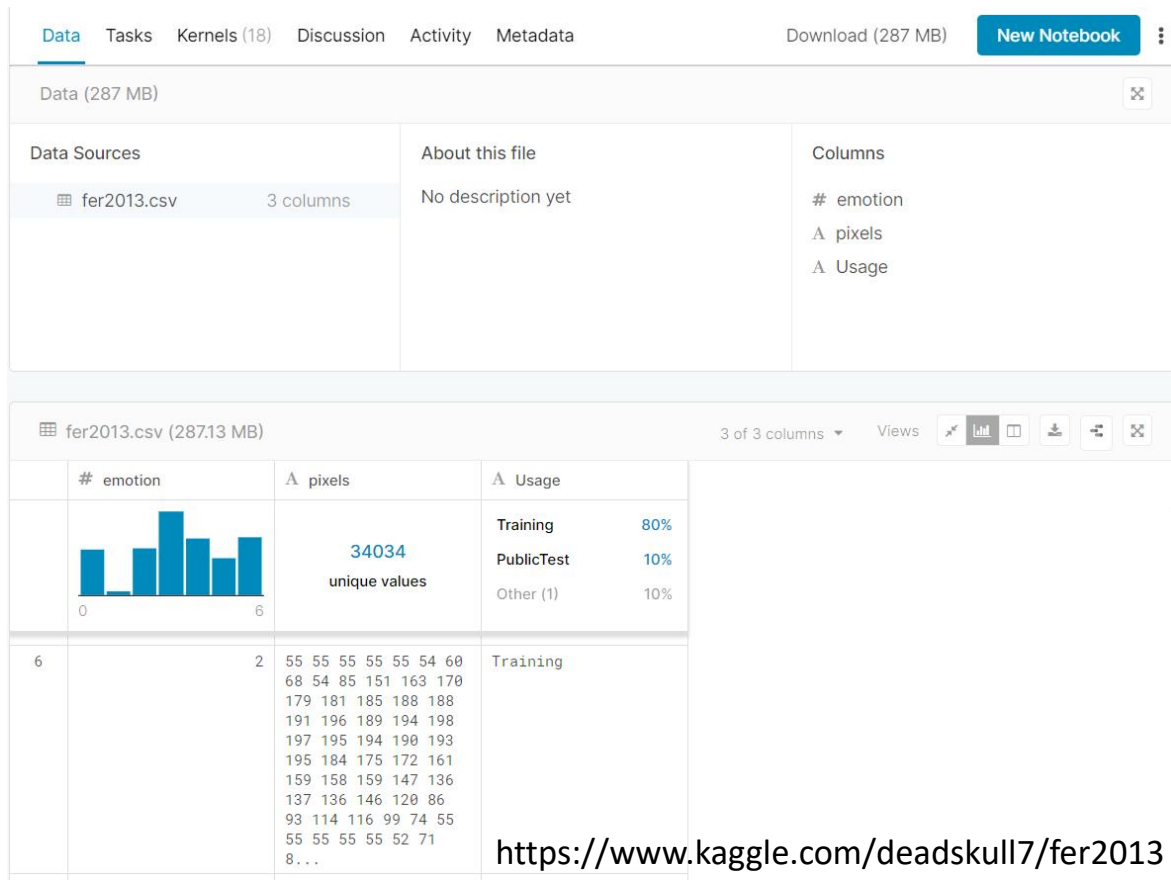
4 属性识别

③数据集介绍

FER13 Dataset

数据集并没有直接给出图片，而是将表情、图片数据、用途的数据保存到csv文件中：

第一张图是csv文件的开头，第一行是表头，说明每列数据的含义，第一列表示表情标签，第二列即为图片数据，这里是原始的图片数据，最后一列为用途。



<https://www.kaggle.com/deadskull7/fer2013>

4 属性识别

③数据集介绍

FER13 Dataset

数据集并没有直接给出图片，而是将表情、图片数据、用途的数据保存到csv文件中：

第一张图是csv文件的开头，第一行是表头，说明每列数据的含义，第一列表示表情标签，第二列即为图片数据，这里是原始的图片数据，最后一列为用途。

	A	B	C
1	emotion	pixels	Usage
2	0	70 80 82 72 58 58 60 63 54 58 60 48 89 115 121 119 1	Training
3	0	151 150 147 155 148 133 111 140 170 174 182 154 15	Training
4	2	231 212 156 164 174 138 161 173 182 200 106 38 39	Training
5	4	24 32 36 30 32 23 19 20 30 41 21 22 32 34 21 19 43 5	Training
6	6	4 0 0 0 0 0 0 0 0 0 0 3 15 23 28 48 50 58 84 115 127	Training
7	2	55 55 55 55 55 54 60 68 54 85 151 163 170 179 181 1	Training
8	4	20 17 19 21 25 38 42 42 46 54 56 62 63 66 82 108 118	Training
9	3	77 78 79 79 78 75 60 55 47 48 58 73 77 79 57 50 37 4	Training
10	3	85 84 90 121 101 102 133 153 153 169 177 189 195 1	Training
11	2	255 254 255 254 254 179 122 107 95 124 149 150 169	Training

4 属性识别

③表情检测

FER13 Dataset

我们的目标是现在将这个.csv文件转换为HDF5格式，这样我们可以更轻松地在其上面训练卷积神经网络。

解压缩fer2013.tar.gz文件后，我为以下文件设置了以下目录结构：

项目：

```
--- fer2013
| |--- checkpoints
| |--- fer2013
| |--- hdf5
| |--- output
```

	A	B	C
1	emotion	pixels	Usage
2	0	70 80 82 72 58 58 60 63 54 58 60 48 89 115 121 119 1	Training
3	0	151 150 147 155 148 133 111 140 170 174 182 154 15	Training
4	2	231 212 156 164 174 138 161 173 182 200 106 38 39	Training
5	4	24 32 36 30 32 23 19 20 30 41 21 22 32 34 21 19 43 5	Training
6	6	4 0 0 0 0 0 0 0 0 0 0 3 15 23 28 48 50 58 84 115 127	Training
7	2	55 55 55 55 55 54 60 68 54 85 151 163 170 179 181 1	Training
8	4	20 17 19 21 25 38 42 42 46 54 56 62 63 66 82 108 118	Training
9	3	77 78 79 79 78 75 60 55 47 48 58 73 77 79 57 50 37 4	Training
10	3	85 84 90 121 101 102 133 153 153 169 177 189 195 1	Training
11	2	255 254 255 254 254 179 122 107 95 124 149 150 169	Training

4 属性识别

③表情检测

在fer2013目录中，包含了未解压的
fer2013.tar.gz的内容。

文件（即数据集本身）。hdf5目录将包含训练，
测试和验证集。最后，我们将所有训练日志/
图保存到输出目录。

```
--- fer2013
|         |--- checkpoints
|         |--- fer2013
|         |--- hdf5
|         |--- output
```

4 属性识别

③表情检测

创建一个名为config的目录，并将其转换为Python子模块，方法是在其中放置__init__.py,字段名为emotion_config.py 的文件是我们将存储所有配置变量的位置，包括输入数据集的路径，输出HDF5文件和批处理大小。

build_dataset.py将负责提取fer2013.csv数据集文件，并输出一组HDF5文件；每个训练，验证和测试分组中分别一个。

将使用train_recognizer.py训练CNN以识别各种情绪。同样，test_recognizer.py脚本将用于评估CNN的性能。

```
--- emotion_recognition
|   |--- config
|   |   |--- __init__.py
|   |   |--- emotion_config.py
|   |--- build_dataset.py
|   |--- emotion_detector.py
|   |--- haarcascade_frontface_default.xml
|   |--- test_recognizer.py
|   |--- train_recognizer.py
```

4 属性识别

③表情检测

对模型的准确性感到满意后，将继续motion_detector.py脚本可以：

- 1.实时检测面部（如微笑检测器）。
- 2.应用我们的CNN识别最主要的情绪并显示每种情绪的概率分布。

最重要的是，该CNN能够在我们的设备上实时运行。

我们继续并立即查看motion_config.py文件：

```
1  # import the necessary packages
2  from os import path
3
4  # define the base path to the emotion dataset
5  BASE_PATH = "../datasets/fer2013"
6
7  # use the base path to define the path to the input emotions file
8  INPUT_PATH = path.sep.join([BASE_PATH, "fer2013/fer2013.csv"])
```


4 属性识别

③表情检测

在第5行中，我们将BASE_PATH定义到我们的根项目目录。这个位置是输入数据集FER13，输出数据集文件HDF5，日志和图一起的所在地。

```
1  # import the necessary packages
2  from os import path
3
4  # define the base path to the emotion dataset
5  BASE_PATH = "../datasets/fer2013"
6
7  # use the base path to define the path to the input emotions file
8  INPUT_PATH = path.sep.join([BASE_PATH, "fer2013/fer2013.csv"])
9
10 # define the number of classes (set to 6 if you are ignoring the
11 # "disgust" class)
12 # NUM_CLASSES = 7
13 NUM_CLASSES = 6
```

4 属性识别

③表情检测

FER13一共有七个类别：生气，**厌恶**，恐惧，快乐，悲伤，惊奇和中性。但是，“厌恶”与其他表情之间存在严重失衡，因为它只有**113**张图片样本（每个类其余的样本超过**1,000**张）。建议将“厌恶”和“愤怒”合并为一个类（因为情绪在视觉上是相似的），因此将FER13变成了6类问题。

由于我们会将fer2013.csv文件转换为一系列用于训练，验证和测试的HDF5数据集，我们需要

定义这些输出HDF5文件的路径

```
15 # define the path to the output training, validation, and testing
16 # HDF5 files
17 TRAIN_HDF5 = path.sep.join([BASE_PATH, "hdf5/train.hdf5"])
18 VAL_HDF5 = path.sep.join([BASE_PATH, "hdf5/val.hdf5"])
19 TEST_HDF5 = path.sep.join([BASE_PATH, "hdf5/test.hdf5"])
```

4 属性识别

③表情检测

最后，在训练CNN和输出目录时，我们将进行初始化和Batch Size设置，并设置输出目录：

```
21 # define the batch size
22 BATCH_SIZE = 128
23
24 # define the path to where output logs will be stored
25 OUTPUT_PATH = path.sep.join([BASE_PATH, "output"])
```

4 属性识别

③表情检测

现在已经创建了配置文件，可以构建数据集了：

```
1  # import the necessary packages
2  from config import emotion_config as config
3  from pyimagesearch.io import HDF5DatasetWriter
4  import numpy as np
5
6  # open the input file for reading (skipping the header), then
7  # initialize the list of data and labels for the training,
8  # validation, and testing sets
9  print("[INFO] loading input data...")
10 f = open(config.INPUT_PATH)
11 f.__next__() # f.next() for Python 2.7
12 (trainImages, trainLabels) = ([], [])
13 (vallImages, vallLabels) = ([], [])
14 (testImages, testLabels) = ([], [])
```

4 属性识别

③表情检测

在第2行中，我们导入刚刚创建的motion_config文件。

第3行，导入HDF5DatasetWriter，使用它来转换输入的图像到HDF5数据集。

第10行打开指向输入fer2013.csv文件的指针。通过调用文件的.next方法指针，我们可以跳到下一行，从而可以跳过CSV文件的标题。

然后12-14行分别为训练，验证和测试集初始化图像和标签列表。

```
9  print("[INFO] loading input data...")
10 f = open(config.INPUT_PATH)
11 f.__next__() # f.next() for Python 2.7
12 (trainImages, trainLabels) = ([], [])
13 (valImages, valLabels) = ([], [])
14 (testImages, testLabels) = ([], [])
```

4 属性识别

③表情检测

在第17行，我们开始循环遍历输入文件中的每一行。

第19行采用该行，将其拆分为图像标签的三元组，图像的原始像素强度以及使用情况（即训练，测试或验证）。默认情况下，我们假设将FER13视为7类分类问题；但是，如果我们希望将愤怒和厌恶融合在一起分类（第24行），我们需要将厌恶标签从1更改为0（第26和27行）。

```
16 # loop over the rows in the input file
17 for row in f:
18     # extract the label, image, and usage from the row
19     (label, image, usage) = row.strip().split(",")
20     label = int(label)
```

4 属性识别

③表情检测

```
16 # loop over the rows in the input file
17 for row in f:
18     # extract the label, image, and usage from the row
19     (label, image, usage) = row.strip().split(",")
20     label = int(label)
21
22     # if we are ignoring the "disgust" class there will be 6 total
23     # class labels instead of 7
24     if config.NUM_CLASSES == 6:
25         # merge together the "anger" and "disgust" classes
26         if label == 1:
27             label = 0
28
29         # if label has a value greater than zero, subtract one from
30         # it to make all labels sequential (not required, but helps
31         # when interpreting results)
32         if label > 0:
33             label -= 1
```

应从第32行和第33行的每个标签中减去1，以确保每个类标签为连续的，不需要此减法，但在解释我们的结果时会有所帮助。

4 属性识别

③表情检测

此时，我们的图像只是一串整数。我们需要把这个字符串，分成一个列表，将其转换为无符号的8位整数数据类型，并将其整形为 48×48 灰度图像：

请记住，每个图像列都是2304个整数的列表。这2304个整数代表正方形 48×48 图像。我们可以在第38行执行此重塑。

```
35     # reshape the flattened pixel list into a 48x48 (grayscale)
36     # image
37     image = np.array(image.split(" "), dtype="uint8")
38     image = image.reshape((48, 48))
```

4 属性识别

③表情检测

```
40         # check if we are examining a training image
41         if usage == "Training":
42             trainImages.append(image)
43             trainLabels.append(label)
44
45         # check if this is a validation image
46         elif usage == "PrivateTest":
47             valImages.append(image)
48             valLabels.append(label)
49
50         # otherwise, this must be a testing image
51         else:
52             testImages.append(image)
53             testLabels.append(label)
```

4 属性识别

③表情检测

在这里，我们只是初始化数据集列表。列表中的每个条目都是原始的3元组图像，标签和输出HDF5路径。

最后一步是遍历每个训练，验证和测试集：

```
55 # construct a list pairing the training, validation, and testing
56 # images along with their corresponding labels and output HDF5
57 # files
58 datasets = [
59     (trainImages, trainLabels, config.TRAIN_HDF5),
60     (valImages, valLabels, config.VAL_HDF5),
61     (testImages, testLabels, config.TEST_HDF5)]
```

4 属性识别

③表情检测第67行以数据集实例化HDF5DatasetWriter。然后，我们将图像和标签以HDF5格式写入磁盘（第70和71行）。

```
63 # loop over the dataset tuples
64 for (images, labels, outputPath) in datasets:
65     # create HDF5 writer
66     print("[INFO] building {}".format(outputPath))
67     writer = HDF5DatasetWriter((len(images), 48, 48), outputPath)
68
69     # loop over the image and add them to the dataset
70     for (image, label) in zip(images, labels):
71         writer.add([image], [label])
72
73     # close the HDF5 writer
74     writer.close()
75
76 # close the input file
77 f.close()
```

4 属性识别

③表情检测

```
$ python build_dataset.py
[INFO] loading input data...
[INFO] building /raid/datasets/fer2013/hdf5/train.hdf5...
[INFO] building /raid/datasets/fer2013/hdf5/val.hdf5...
[INFO] building /raid/datasets/fer2013/hdf5/test.hdf5...

$ ls -l fer2013/hdf5
total 646268
-rw-rw-r-- 1 adrian adrian 66183304 Aug 29 08:25 test.hdf5
-rw-rw-r-- 1 adrian adrian 529396104 Aug 29 08:25 train.hdf5
-rw-rw-r-- 1 adrian adrian 66183304 Aug 29 08:25 val.hdf5
```

4 属性识别

③表情检测

Layer Type	Output Size	Filter Size / Stride
INPUT IMAGE	$48 \times 48 \times 1$	
CONV	$48 \times 48 \times 32$	$3 \times 3, K = 32$
CONV	$48 \times 48 \times 32$	$3 \times 3, K = 32$
POOL	$24 \times 24 \times 32$	2×2
CONV	$24 \times 24 \times 64$	$3 \times 3, K = 64$
CONV	$24 \times 24 \times 64$	$3 \times 3, K = 64$
POOL	$12 \times 12 \times 64$	2×2
CONV	$12 \times 12 \times 128$	$3 \times 3, K = 128$
CONV	$12 \times 12 \times 128$	$3 \times 3, K = 128$
POOL	$6 \times 6 \times 128$	2×2
FC	64	
FC	64	
FC	6	
SOFTMAX	6	

4 属性识别

③表情检测

我们将要实现的用于识别各种情绪和面部表情的网络是受VGG网络家族的启发：

- 1.网络中的CONV层将仅为 3×3 。

- 2.随着网络的加深，我们会将每个CONV层学习的过滤器数量增加一倍。

为了帮助网络训练，我们将在第8章中应用从VGG和ImageNet实验获得的一些先验知识：

- 1.我们使用MSRA (He等人)的方法初始化CONV和FC层，这样做将使我们的网络学习更快。

- 2.由于已证明ELU和PReLU可以提高所有分类的分类准确性，在我们的实验中，我们仅以ELU而非ReLU开始。

- 3.表中包含了名为EmotionVGGNet的网络摘要。每次CONV层之后，我们将应用激活，然后进行批量归一化（将这些层排除在表外以节省空间）。

4 属性识别

③表情检测

emotionvggnet.py

```
1  # import the necessary packages
2  from tensorflow.keras.models import Sequential
3  from tensorflow.keras.layers import BatchNormalization
4  from tensorflow.keras.layers import Conv2D
5  from tensorflow.keras.layers import MaxPooling2D
6  from tensorflow.keras.layers import ELU
7  from tensorflow.keras.layers import Activation
8  from tensorflow.keras.layers import Flatten
9  from tensorflow.keras.layers import Dropout
10 from tensorflow.keras.layers import Dense
11 from tensorflow.keras import backend as K
```

4 属性识别

③表情检测

从那里，我们可以定义EmotionVGGNet的构建方法：

```
13 class EmotionVGGNet:
14     @staticmethod
15     def build(width, height, depth, classes):
16         # initialize the model along with the input shape to be
17         # "channels last" and the channels dimension itself
18         model = Sequential()
19         inputShape = (height, width, depth)
20         chanDim = -1
21
22         # if we are using "channels first", update the input shape
23         # and channels dimension
24         if K.image_data_format() == "channels_first":
25             inputShape = (depth, height, width)
26             chanDim = 1
```

第18行初始化模型对象。

假设“最后一个通道”的顺序（第19行和第20行），但是如果改用“通道优先”排序，我们还需要更新输入形状大小和通道（24-26）。

4 属性识别

③表情检测 第一CONV层将学习32个 3×3 卷积核。然后，我们将应用ELU激活通过批量归一化。

同样，第二个CONV层应用相同的模式，学习32个 3×3 卷积核，然后进行ELU和批量归一化。

```
28 # Block 1: CONV -> BN -> ELU -> CONV -> RELU -> POOL
29 # layer set
30 model.add(Conv2D(32, (3, 3), padding="same",
31                 kernel_initializer="he_normal", input_shape=inputShape))
32 model.add(ELU())
33 model.add(BatchNormalization(axis=chanDim))
34 model.add(Conv2D(32, (3, 3), kernel_initializer="he_normal",
35                 padding="same"))
36 model.add(ELU())
37 model.add(BatchNormalization(axis=chanDim))
38 model.add(MaxPooling2D(pool_size=(2, 2)))
39 model.add(Dropout(0.25))
```

4 属性识别

③表情检测

EmotionVGGNet中的第二个块与第一个块相同，只是现在将CONV层中的卷积核数为64，而不是32：

```
41     # Block #2: second CONV => RELU => CONV => RELU => POOL
42     # layer set
43     model.add(Conv2D(64, (3, 3), kernel_initializer="he_normal",
44                     padding="same"))
45     model.add(ELU())
46     model.add(BatchNormalization(axis=chanDim))
47     model.add(Conv2D(64, (3, 3), kernel_initializer="he_normal",
48                     padding="same"))
49     model.add(ELU())
50     model.add(BatchNormalization(axis=chanDim))
51     model.add(MaxPooling2D(pool_size=(2, 2)))
52     model.add(Dropout(0.25))
```

4 属性识别

③表情检测

转到第三块，再次应用相同的模式，增加了卷积核从64到128,随着CNN的深入，我们需要学习的特征越多，需要的卷积核也越多：

```
54 # Block #3: third CONV => RELU => CONV => RELU => POOL
55 # layer set
56 model.add(Conv2D(128, (3, 3), kernel_initializer="he_normal",
57                 padding="same"))
58 model.add(ELU())
59 model.add(BatchNormalization(axis=chanDim))
60 model.add(Conv2D(128, (3, 3), kernel_initializer="he_normal",
61                 padding="same"))
62 model.add(ELU())
63 model.add(BatchNormalization(axis=chanDim))
64 model.add(MaxPooling2D(pool_size=(2, 2)))
65 model.add(Dropout(0.25))
```

4 属性识别

③表情检测

接下来，我们需要构造第一个完全连接层，在这里，学习了64个隐藏节点，然后应用ELU激活功能并进行批量正则化。后续将以相同的方式应用第二个FC层

```
67         # Block #4: first set of FC => RELU layers
68         model.add(Flatten())
69         model.add(Dense(64, kernel_initializer="he_normal"))
70         model.add(ELU())
71         model.add(BatchNormalization())
72         model.add(Dropout(0.5))
73
74         # Block #6: second set of FC => RELU layers
75         model.add(Dense(64, kernel_initializer="he_normal"))
76         model.add(ELU())
77         model.add(BatchNormalization())
78         model.add(Dropout(0.5))
```

4 属性识别

③表情检测

最后，我们将在FC层中应用提供的类数以及softmax分类器以获取我们的输出类标签概率：

```
80         # Block #7: softmax classifier
81         model.add(Dense(classes, kernel_initializer="he_normal"))
82         model.add(Activation("softmax"))
83
84         # return the constructed network architecture
85         return model
```


4 属性识别

③表情检测

train_recognizer.py

```
1  # set the matplotlib backend so figures can be saved in the background
2  import matplotlib
3  matplotlib.use("Agg")
4
5  # import the necessary packages
6  from config import emotion_config as config
7  from pyimagesearch.preprocessing import ImageToArrayPreprocessor
8  from pyimagesearch.callbacks import EpochCheckpoint
9  from pyimagesearch.callbacks import TrainingMonitor
10 from pyimagesearch.io import HDF5DatasetGenerator
11 from pyimagesearch.nn.conv import EmotionVGGNet
12 from tensorflow.keras.preprocessing.image import ImageDataGenerator
13 from tensorflow.keras.optimizers import Adam
14 from tensorflow.keras.models import load_model
15 import tensorflow.keras.backend as K
16 import argparse
17 import os
```

4 属性识别

③表情检测

```
19 # construct the argument parse and parse the arguments
20 ap = argparse.ArgumentParser()
21 ap.add_argument("-c", "--checkpoints", required=True,
22                 help="path to output checkpoint directory")
23 ap.add_argument("-m", "--model", type=str,
24                 help="path to *specific* model checkpoint to load")
25 ap.add_argument("-s", "--start-epoch", type=int, default=0,
26                 help="epoch to restart training at")
27 args = vars(ap.parse_args())
```


4 属性识别

③表情检测

正如预期的那样，我们将在训练集中应用数据增强来帮助减少过度拟合和提高模型的分类精度（第31和32行）。

但是你可能不熟悉 正在将数据增强应用于验证集（第33行）。为什么我们需要将数据增强应用于验证集？

```
29 # construct the training and testing image generators for data
30 # augmentation, then initialize the image preprocessor
31 trainAug = ImageDataGenerator(rotation_range=10, zoom_range=0.1,
32     horizontal_flip=True, rescale=1 / 255.0, fill_mode="nearest")
33 valAug = ImageDataGenerator(rescale=1 / 255.0)
34 iap = ImageToArrayPreprocessor()
```

4 属性识别

③表情检测

答案在于`rescale`属性（也是训练数据增强器的一部分）。回想我们将`fer2013.csv`文件转换为HDF5数据集。我们把这些图像作为原始的、未归一化的RGB图像，这意味着像素值被允许存在于`[0, 255]`范围内。然而，通常的做法是：

(1)执行平均归一化 (2)缩放像素到一个更狭窄的变化区间。

幸运的是，Keras提供的图像数据生成器类可以自动为我们执行此缩放。我们只需要提供`1/255`的缩放比，这样每幅图像都将以此比率为倍数，从而将像素缩小到`[0, 1]`。

```
29 # construct the training and testing image generators for data
30 # augmentation, then initialize the image preprocessor
31 trainAug = ImageDataGenerator(rotation_range=10, zoom_range=0.1,
32                               horizontal_flip=True, rescale=1 / 255.0, fill_mode="nearest")
33 valAug = ImageDataGenerator(rescale=1 / 255.0)
34 iap = ImageToArrayPreprocessor()
```

4 属性识别

③表情检测

初始化训练和验证集的HDF5DatasetGenerator对象后，输入HDF5文件的路径，批处理大小和类数均来自我们的config文件，可以使我们轻松更改这些参数。

```
36 # initialize the training and validation dataset generators
37 trainGen = HDF5DatasetGenerator(config.TRAIN_HDF5, config.BATCH_SIZE,
38     aug=trainAug, preprocessors=[iap], classes=config.NUM_CLASSES)
39 valGen = HDF5DatasetGenerator(config.VAL_HDF5, config.BATCH_SIZE,
40     aug=valAug, preprocessors=[iap], classes=config.NUM_CLASSES)
```

4 属性识别

③表情检测

如果没有提供特定的模型检查点，则从第一个epoch开始训练新的模型：

```
42 # if there is no specific model checkpoint supplied, then initialize
43 # the network and compile the model
44 if args["model"] is None:
45     print("[INFO] compiling model...")
46     model = EmotionVGGNet.build(width=48, height=48, depth=1,
47                                 classes=config.NUM_CLASSES)
48     opt = Adam(lr=1e-3)
49     model.compile(loss="categorical_crossentropy", optimizer=opt,
50                  metrics=["accuracy"])
```

4 属性识别

③表情检测

否则，我们将从磁盘加载特定的检查点，更新学习率，并且重新开始训练：

```
52 # otherwise, load the checkpoint from disk
53 else:
54     print("[INFO] loading {}...".format(args["model"]))
55     model = load_model(args["model"])
56
57     # update the learning rate
58     print("[INFO] old learning rate: {}".format(
59         K.get_value(model.optimizer.lr)))
60     K.set_value(model.optimizer.lr, 1e-3)
61     print("[INFO] new learning rate: {}".format(
62         K.get_value(model.optimizer.lr)))
```


4 属性识别

③表情检测

在我们开始训练过程之前，我们必须构建一个callbacks列表，用于将检查点序持久化到磁盘，并在一段时间内记录准确性/损失：

```
64 # construct the set of callbacks
65 figPath = os.path.sep.join([config.OUTPUT_PATH,
66                             "vggnet_emotion.png"])
67 jsonPath = os.path.sep.join([config.OUTPUT_PATH,
68                              "vggnet_emotion.json"])
69 callbacks = [
70     EpochCheckpoint(args["checkpoints"], every=5,
71                    startAt=args["start_epoch"]),
72     TrainingMonitor(figPath, jsonPath=jsonPath,
73                    startAt=args["start_epoch"])]
```


4 属性识别

③表情检测

```
75 # train the network
76 model.fit_generator(
77     trainGen.generator(),
78     steps_per_epoch=trainGen.numImages // config.BATCH_SIZE,
79     validation_data=valGen.generator(),
80     validation_steps=valGen.numImages // config.BATCH_SIZE,
81     epochs=15,
82     max_queue_size=config.BATCH_SIZE * 2,
83     callbacks=callbacks, verbose=1)
84
85 # close the databases
86 trainGen.close()
87 valGen.close()
```

Epoch	Learning Rate
1 – 20	$1e-2$
21 – 40	$1e-3$
41 – 60	$1e-4$

4 属性识别

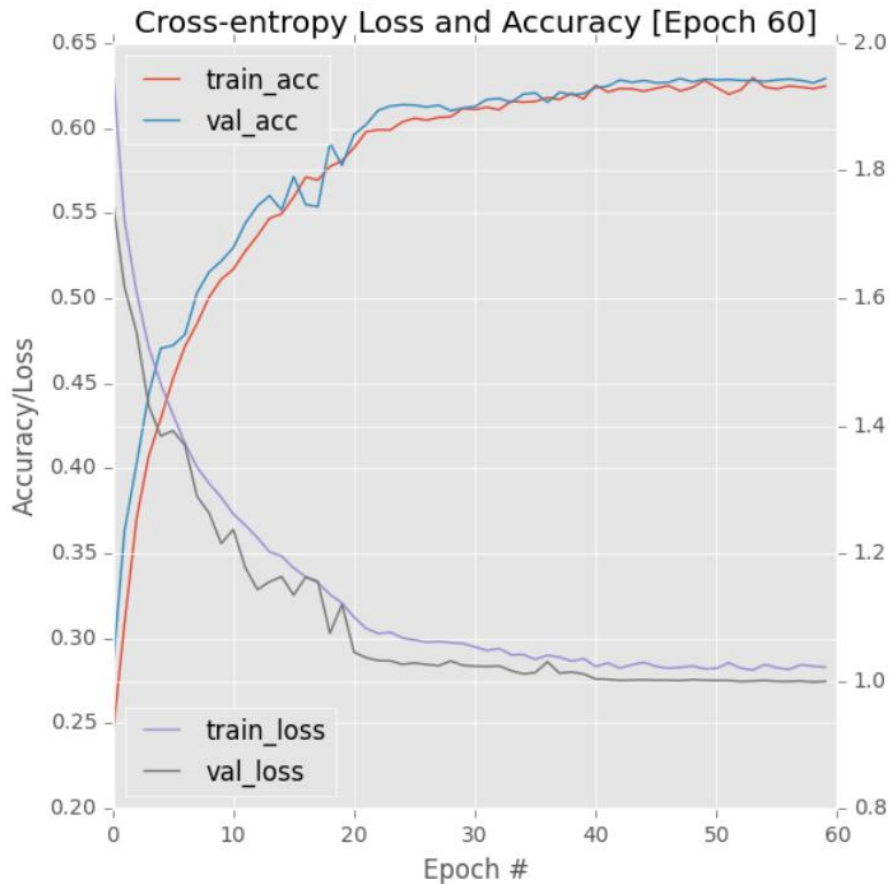
④参数优化1

从SGD优化器开始，其基本学习率为 $1e-2$ ，动量项为0.9，并且应用了Nesterov加速度。（默认）

Xavier / Glorot初始化方法用于初始化CONV和FC层中的权重。此外，唯一的数据扩充功能是水平翻转-没有其他数据增扩方式（例如旋转缩放等）。

```
$ python train_recognizer.py --checkpoints checkpoints
```

Epoch	Learning Rate
1 – 20	$1e-2$
21 – 40	$1e-3$
41 – 60	$1e-4$



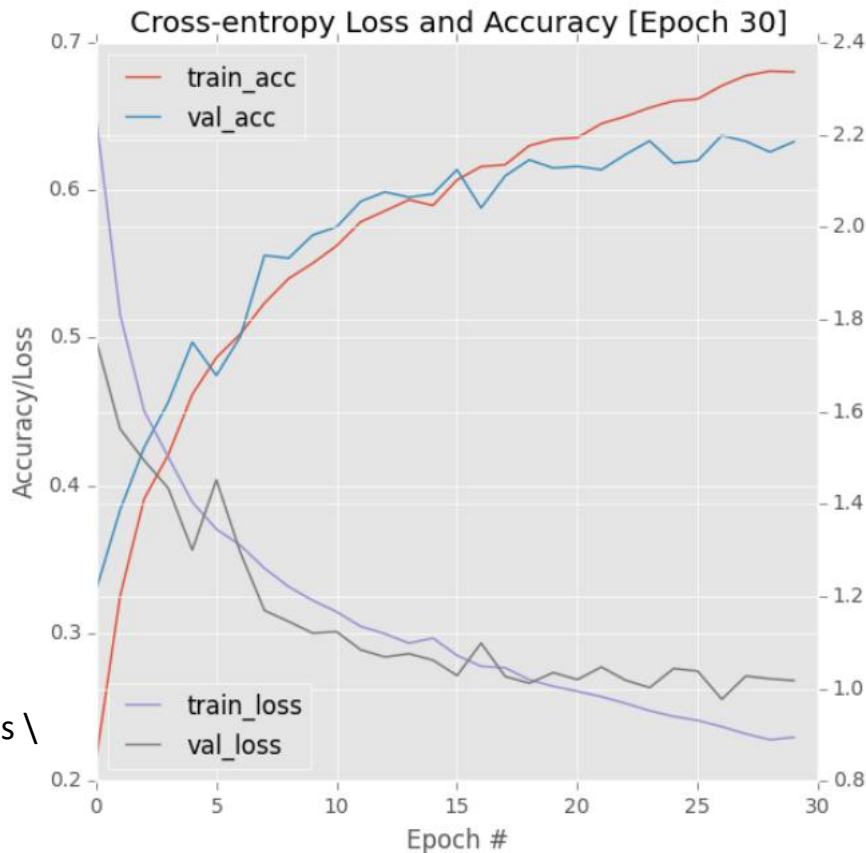
4 属性识别

④参数优化2

鉴于SGD在降低学习率时导致学习停滞，我决定换掉它，而使用基本学习率为 $1e-3$ 的Adam代替SGD。除了调整优化器之外，此实验其他参数与第一个实验相同。

在第30个epoch，开始注意到训练损失与验证损失之间的巨大差异，因此停止了训练并将学习率从 $1e-3$ 降低到 $1e-4$ ，然后允许网络再训练15个epoch。

```
$ python train_recognizer.py --checkpoints checkpoints \
--model checkpoints/epoch_30.hdf5 --start-epoch 30
```

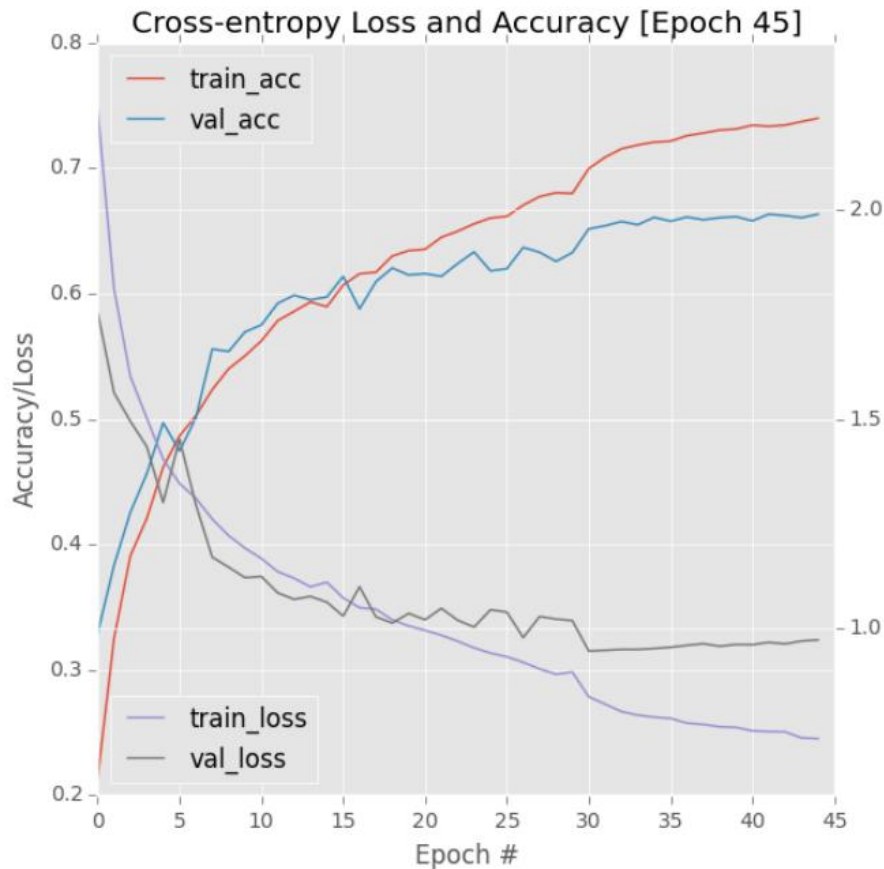


4 属性识别

④参数优化2

但是，结果并不理想。如我们所见，显然存在过度拟合-训练损失继续减少，而验证损失不仅停滞不前，而且还在继续增加。

话虽如此，在第45个epoch结束时，该网络仍能获得66.34%的准确性，比SGD好。如果我可以找到抑制过度拟合的方法，那么Adam优化器方法将在这种情况下可能表现很好。



4 属性识别

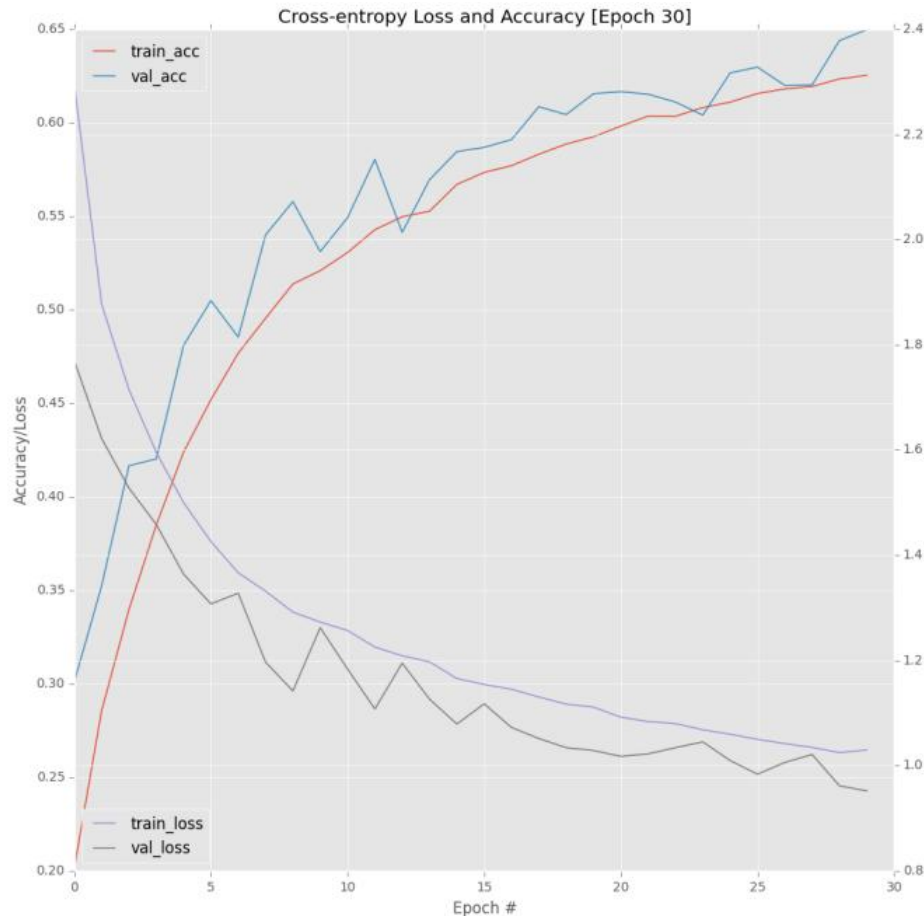
④参数优化3

解决过拟合的一种常见方法是收集更多代表您的验证的训练数据和测试集。

但是，由于FER2013数据集已预先编译，收集其他数据是不可能的。相反，我们可以应用数据增强来帮助减少过度拟合。

在第三个实验中，我保留了Adam优化器，但还添加了随机旋转范围10度，zoom range为0.1等数据增强方法（zoom_range: 浮点数或形如[lower,upper]的列表，随机缩放的幅度，若为浮点数，则相当于[lower,upper] = [1 - zoom_range, 1+zoom_range]）。

有了新的数据扩充方案，我重复了第二个实验：

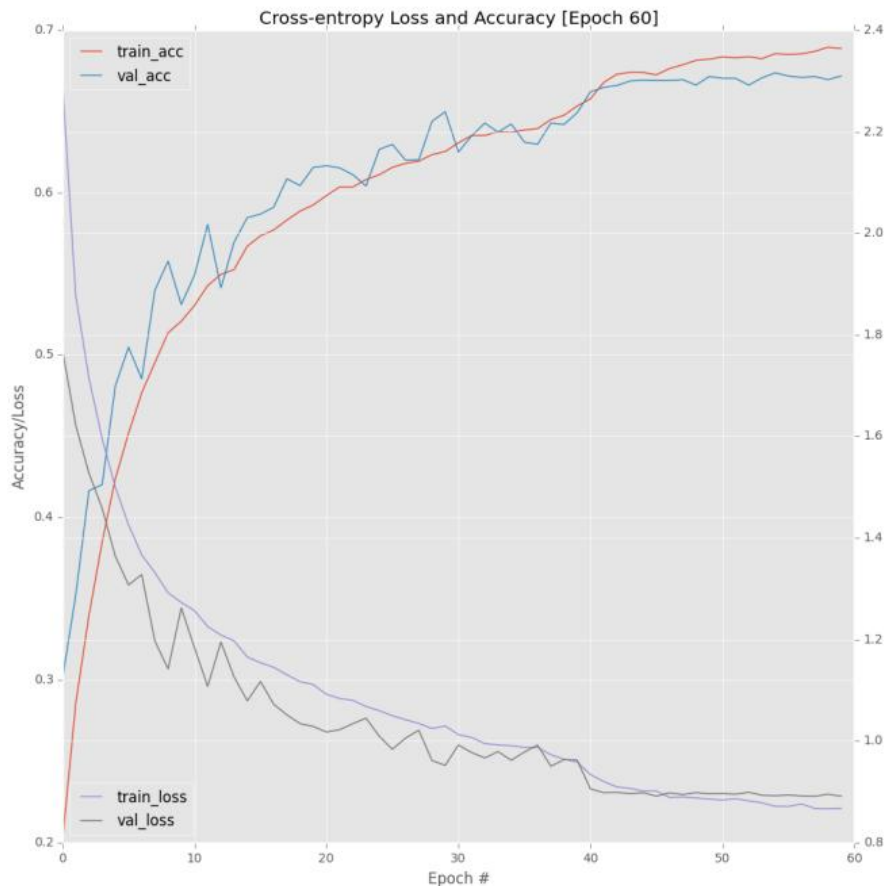


4 属性识别

④参数优化3

如下图所示，约在epoch35饱和和开始出现。此时，我停止训练，降低了Adam的学习率，把它从 $1e-3$ 降至 $1e-4$ ，并恢复训练：这个过程先是导致了精度上的损失，后来精度恢复上升，所以我再次在60时停止训练，将学习率从 $1e-4$ 降低到 $1e-5$ ，并恢复训练再来15个epoch，共75epoch。

```
$ python train_recognizer.py --checkpoints checkpoints \
--model checkpoints/epoch_40.hdf5 --start-epoch 40
```

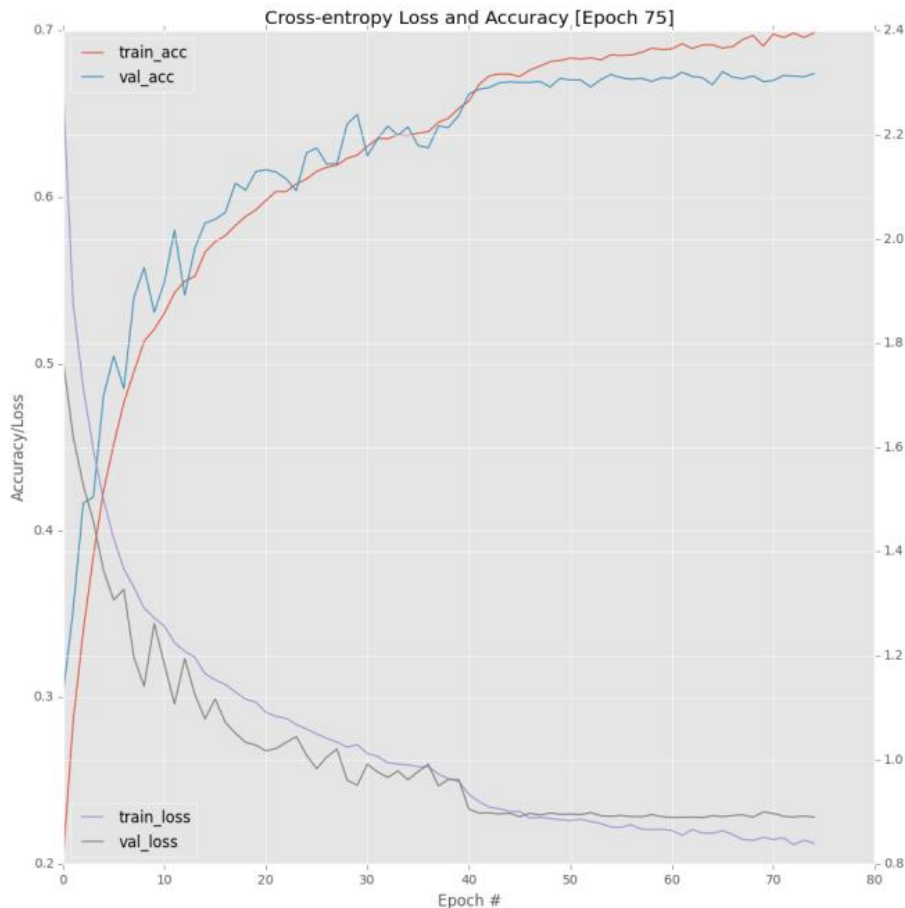


4 属性识别

④参数优化3

如我们所见，我们现在没有过度拟合的风险了-不利之处在于，前epoch45中，我们在准确性方面没有任何重大进步。

综上所述，通过应用数据增强，我们能够稳定学习，减少过度拟合，并允许我们在验证集上达到67.53%的分类精度。



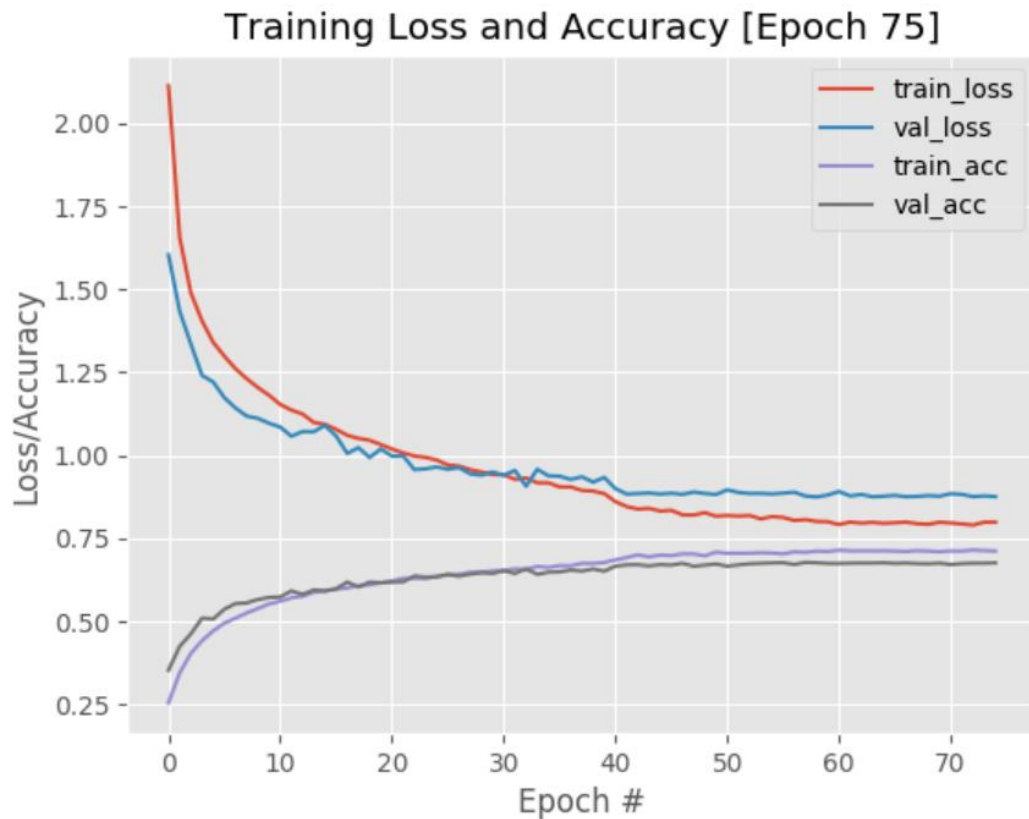
4 属性识别

④参数优化4

在FER2013和EmotionVGGNet的最终实验中，我决定进行一些更改：

1. 我将Xavier / Glorot初始化（Keras使用的默认设置）换成了MSRA / He初始化。
2. 将所有ReLU替换为ELU，以进一步提高准确性。

Epoch	Learning Rate
1 – 40	$1e-3$
40 – 60	$1e-4$
61 – 75	$1e-5$

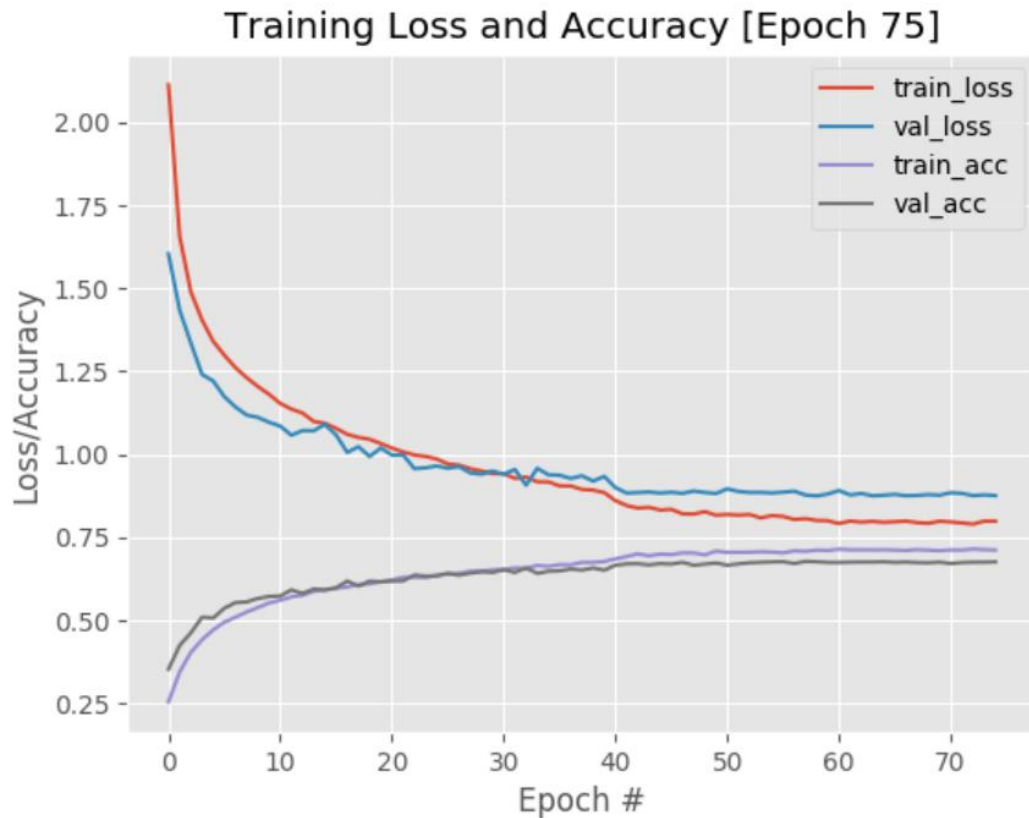


4 属性识别

④参数优化4

在FER2013和EmotionVGGNet的最终实验中，我决定进行一些更改：

3. 鉴于“厌恶”标签引起的数据失衡，我将“愤怒”和“厌恶”合并为一个类别。为了合并这两个类，我需要再次使用运行 `build_dataset.py` 使用其中的 `NUM_CLASSES` 把它设置为六个而不是七个。



4 属性识别

④参数优化4

第2-7行导入了我们所需的Python包。

第2行会导入我们的motion_config，因此我们有权访问我们的项目配置变量。

HDF5DatasetGenerator将需要访问测试集（第4行）。

我们还将导入ImageDataGenerator，以便我们可以缩放FER2013中的图像降低到[0,1]范围。

```
1  # import the necessary packages
2  from config import emotion_config as config
3  from pyimagesearch.preprocessing import ImageToArrayPreprocessor
4  from pyimagesearch.io import HDF5DatasetGenerator
5  from tensorflow.keras.preprocessing.image import ImageDataGenerator
6  from tensorflow.keras.models import load_model
7  import argparse
```

4 属性识别

④参数优化4

在执行第20和21行前，需要确保：

- （1）我们的数据已进行重新缩放，
- （2）已将我们的图像应用于Keras兼容数组转换器。

```
10 ap = argparse.ArgumentParser()
11 ap.add_argument("-m", "--model", type=str,
12                 help="path to model checkpoint to load")
13 args = vars(ap.parse_args())
14
15 # initialize the testing data generator and image preprocessor
16 testAug = ImageDataGenerator(rescale=1 / 255.0)
17 iap = ImageToArrayPreprocessor()
18
19 # initialize the testing dataset generator
20 testGen = HDF5DatasetGenerator(config.TEST_HDF5, config.BATCH_SIZE,
21                                aug=testAug, preprocessors=[iap], classes=config.NUM_CLASSES)
```

4 属性识别

④参数优化4

```
23 # load the model from disk
24 print("[INFO] loading {}...".format(args["model"]))
25 model = load_model(args["model"])
26
27 # evaluate the network
28 (loss, acc) = model.evaluate_generator(
29     testGen.generator(),
30     steps=testGen.numImages // config.BATCH_SIZE,
31     max_queue_size=config.BATCH_SIZE * 2)
32 print("[INFO] accuracy: {:.2f}".format(acc * 100))
33
34 # close the testing database
35 testGen.close()
```


4 属性识别

④参数优化4

```
$ python test_recognizer.py --model checkpoints/epoch_75.hdf5  
[INFO] loading checkpoints/epoch_75.hdf5...  
[INFO] accuracy: 66.96
```

4 属性识别

⑤模型应用

emotion_detector.py

```
1  # import the necessary packages
2  from tensorflow.keras.preprocessing.image import img_to_array
3  from tensorflow.keras.models import load_model
4  import numpy as np
5  import argparse
6  import imutils
7  import cv2
8
9  # construct the argument parse and parse the arguments
10 ap = argparse.ArgumentParser()
11 ap.add_argument("-c", "--cascade", required=True,
12                 help="path to where the face cascade resides")
13 ap.add_argument("-m", "--model", required=True,
14                 help="path to pre-trained emotion detector CNN")
15 ap.add_argument("-v", "--video",
16                 help="path to the (optional) video file")
17 args = vars(ap.parse_args())
```

4 属性识别

⑤模型应用

从那里，我们加载人脸检测级联，情感检测CNN并初始化

我们的CNN可以预测的情感标签列表如23行所示：

```
19 # load the face detector cascade, emotion detection CNN, then define
20 # the list of emotion labels
21 detector = cv2.CascadeClassifier(args["cascade"])
22 model = load_model(args["model"])
23 EMOTIONS = ["angry", "scared", "happy", "sad", "surprised",
24             "neutral"]
```

4 属性识别

⑤模型应用

cv2.VideoCapture对象实例化:

(1) 访问网络摄像头或 (2) 从视频文件读取:

```
26 # if a video path was not supplied, grab the reference to the webcam
27 if not args.get("video", False):
28     camera = cv2.VideoCapture(0)
29
30 # otherwise, load the video
31 else:
32     camera = cv2.VideoCapture(args["video"])
```

4 属性识别

⑤模型应用

我们现在准备从视频指针开始在帧上循环：

第37行从视频流中读取下一帧，如果未抓取框架（即设置为False），如果我们正在从视频流中读取帧，已经到达文件末尾时，因此我们应该从循环中中断（第41和42行）。

```
34  # keep looping
35  while True:
36      # grab the current frame
37      (grabbed, frame) = camera.read()
38
39      # if we are viewing a video and we did not grab a
40      # frame, then we have reached the end of the video
41      if args.get("video") and not grabbed:
42          break
```

4 属性识别

⑤模型应用

我们初始化一个空的NumPy画布（第50行），其宽度为300px，高度为200px。我们将使用画布绘制CNN预测的概率分布，使我们能够可视化情感的概率和混合性。然后，第55-57行使用OpenCV的预先训练的Haar级联来检测帧中的人脸。

```

44 # resize the frame and convert it to grayscale
45 frame = imutils.resize(frame, width=300)
46 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
47
48 # initialize the canvas for the visualization, then clone
49 # the frame so we can draw on it
50 canvas = np.zeros((220, 300, 3), dtype="uint8")
51 frameClone = frame.copy()
52
53 # detect faces in the input frame, then clone the frame so that
54 # we can draw on it
55 rects = detector.detectMultiScale(gray, scaleFactor=1.1,
56 minNeighbors=5, minSize=(30, 30),
57 flags=cv2.CASCADE_SCALE_IMAGE)

```


4 属性识别

⑤模型应用

第60行确保在帧中至少检测到一张脸。假设至少有检测到一张脸后，我们根据边界框的大小对边界框列表`rect`进行排序，在列表的最前面放大脸（第62和64行）。

```
59         # ensure at least one face was found before continuing
60         if len(rects) > 0:
61             # determine the largest face area
62             rect = sorted(rects, reverse=True,
63                           key=lambda x: (x[2] - x[0]) * (x[3] - x[1]))[0]
64             (fX, fY, fW, fH) = rect
65
66         # extract the face ROI from the image, then pre-process
67         # it for the network
68         roi = gray[fY:fY + fH, fX:fX + fW]
69
70         roi = cv2.resize(roi, (48, 48))
71         roi = roi.astype("float") / 255.0
72         roi = img_to_array(roi)
73         roi = np.expand_dims(roi, axis=0)
```

我们通过将ROI调整为固定的
48×48像素(EmotionVGGNet所
需的输入大小)来预处理ROI。

从那里，我们将ROI转换为浮
点数据 类型，将其缩放到范围
[0, 1]，并将其转换为与Keras
兼容的数组（第69-72行）。

4 属性识别

⑤模型应用

第76行调用模型的预测方法，该方法返回预测的类标签概率。因此，标签是关联概率最大的标签（第77行）。但是，由于人的面部表情通常是多种情绪的结合，因此需要查看标签的概率分布。

在80行上遍历标签和相关概率。86行和87行绘制条形图其中每个条形宽度与预测的类别标签概率成比例。然后画88-90行画布上标签的名称。

```
74 # make a prediction on the ROI, then lookup the class
75 # label
76 preds = model.predict(roi)[0]
77 label = EMOTIONS[preds.argmax()]
78
79 # loop over the labels + probabilities and draw them
80 for (i, (emotion, prob)) in enumerate(zip(EMOTIONS, preds)):
81     # construct the label text
82     text = "{:}: {:.2f}%".format(emotion, prob * 100)
83
84     # draw the label + probability bar on the canvas
85     w = int(prob * 300)
86     cv2.rectangle(canvas, (5, (i * 35) + 5),
87                    (w, (i * 35) + 35), (0, 0, 255), -1)
88     cv2.putText(canvas, text, (10, (i * 35) + 23),
89                  cv2.FONT_HERSHEY_SIMPLEX, 0.45,
90                  (255, 255, 255), 2)
```

4 属性识别

⑤模型应用

```
92         # draw the label on the frame
93         cv2.putText(frameClone, label, (fX, fY - 10),
94                     cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 0, 255), 2)
95         cv2.rectangle(frameClone, (fX, fY), (fX + fW, fY + fH),
96                     (0, 0, 255), 2)
97
98     # show our classifications + probabilities
99     cv2.imshow("Face", frameClone)
100    cv2.imshow("Probabilities", canvas)
101
102    # if the 'q' key is pressed, stop the loop
103    if cv2.waitKey(1) & 0xFF == ord("q"):
104        break
105
106    # cleanup the camera and close any open windows
107    camera.release()
108    cv2.destroyAllWindows()
```

4 属性识别

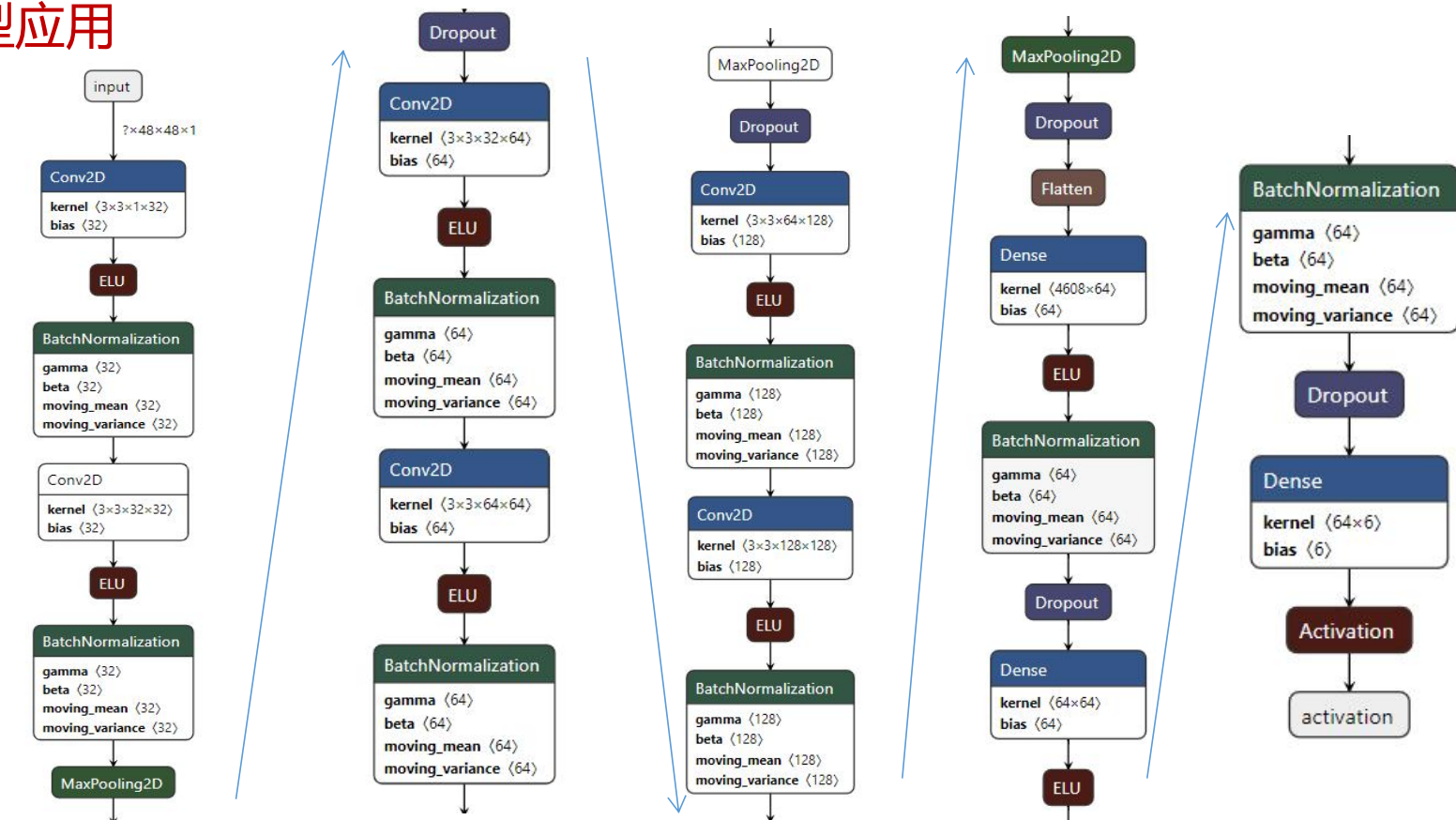
⑤模型应用

```
$ python emotion_detector.py --cascade haarcascade_frontalface_default.xml --model checkpoints/epoch_75.hdf5
```

```
$ python emotion_detector.py --cascade haarcascade_frontalface_default.xml \  
--model checkpoints/epoch_75.hdf5 --video path/to/your/video.mp4
```

4 属性识别

⑤模型应用



4 属性识别

Three windows labeled "Face" show a person's face with a red bounding box and a predicted emotion label above it. Below each face window is a "Probabilities" window showing the predicted emotion and its percentage, along with other emotion percentages.

Left Window: The person is smiling. The predicted emotion is "happy". The "Probabilities" window shows:

- angry: 0.41%
- scored: 0.48%
- happy: 92.66%
- sad: 2.19%
- surprised: 0.20%
- neutral: 4.06%

Middle Window: The person has a neutral expression. The predicted emotion is "neutral". The "Probabilities" window shows:

- angry: 4.60%
- scored: 3.46%
- happy: 1.45%
- sad: 20.30%
- surprised: 0.18%
- neutral: 70.02%

Right Window: The person has an angry expression. The predicted emotion is "angry". The "Probabilities" window shows:

- angry: 43.69%
- scored: 10.77%
- happy: 0.39%
- sad: 30.96%
- surprised: 0.54%
- neutral: 13.66%

